

Huwa 1.0.0 : description technique des fonctions de cryptologie

Annexe technique à la déclaration de fourniture d'un moyen de cryptologie (décret n° 2007-663 du 2 mai 2007).

Rubrique	Valeur
Produit	Huwa, application mobile, version 1.0.0
Identifiant	com.amintt2.huwa (iOS, Android)
Fournisseur	Tahar Touzi, personne physique, France
Nature	Logiciel grand public, gratuit, distribué sous forme binaire (App Store, AltStore PAL, IPA, APK)
Date du document	Octobre 2026

Toutes les informations ci-dessous ont été relevées dans le code source de la version 1.0.0 et dans les versions exactes des bibliothèques résolues par `package-lock.json` (JavaScript) et `Cargo.lock` (Rust).

1. Synthèse

Question	Réponse
Fonctions assurées	Confidentialité (chiffrement de transport, chiffrement des messages privés, chiffrement de la sauvegarde de la phrase de récupération) ; authentification et intégrité (signatures, MAC, hachage)
Algorithmes	Uniquement des algorithmes publics et standard. Aucun algorithme propriétaire ou non publié.
Taille de clé symétrique maximale	256 bits (ChaCha20, XChaCha20, XSalsa20, AES-256)
Asymétrique	Courbe Curve25519 / Edwards25519 (X25519, Ed25519 : clés de 256 bits) ; ECDHE et signatures RSA/ECDSA dans TLS (côté serveur, vérification seulement)
Origine du code cryptographique	Bibliothèques libres tierces embarquées (libsodium, @noble/hashes, rustls/aws-lc-rs, GnuTLS) et services du système d'exploitation (CryptoKit, Security.framework, Android Keystore, piles TLS du système)
Paramétrage par l'utilisateur	Aucun. L'utilisateur ne peut ni choisir, ni modifier, ni désactiver un algorithme ou une taille de clé, ni importer de code cryptographique.
Chiffrement de données arbitraires	Non. L'application ne permet pas de chiffrer des fichiers ou des données au choix de l'utilisateur. Seuls les messages privés, les données de synchronisation de l'application et la phrase de récupération sont protégés.
Algorithmes faibles	SHA-1 est utilisé uniquement pour vérifier l'intégrité des morceaux de fichiers BitTorrent (protocole BitTorrent v1), pas pour la sécurité. Le chiffrement de protocole BitTorrent (MSE/PE, RC4) n'est pas implémenté.

2. Vue d'ensemble des composants

#	Composant	Rôle	Fournisseur du code cryptographique	Présent dans
A	Couche pair à pair (worklet Bare)	Transport chiffré entre appareils, journaux signés, messages privés chiffrés, appairage d'appareils	libsodium 1.0.21 embarqué (via sodium-native 5.1.0 / sodium-universal 5.0.1)	Toutes les versions

#	Composant	Rôle	Fournisseur du code cryptographique	Présent dans
B	Sauvegarde par clé d'accès (passkey)	Chiffrement de la phrase de récupération	HKDF-SHA-256 : @noble/hashes 2.4.0 (embarqué) ; AES-256-GCM : CryptoKit (iOS) via expo-crypto 57.0.3	iOS 18+ uniquement
C	Stockage des secrets	Phrase de récupération et graine locales ; copie synchronisée dans le Trousseau iCloud	Système : Trousseau iOS (Security.framework), Android Keystore via expo-secure-store 57.0.4	Toutes les versions
D	Moteur BitTorrent (Rust)	Clients HTTPS (trackers, proxy de lecture) ; contrôle d'intégrité des morceaux	rustls 0.23.45 avec le fournisseur aws-lc-rs 1.18.1 (aws-lc-sys 0.45.0), reqwest 0.13.5 ; ring 0.17.14 également présent	Versions « complètes » seulement (AltStore PAL, IPA, APK), pas la version App Store
E	Lecteur de secours libmpv (iOS)	Client HTTPS des flux vidéo ; déchiffrement des segments HLS AES-128	GnuTLS 3.8.11 et FFmpeg n8.1.2 embarqués (MPVKit 1.0.0, mpv 0.41.0)	iOS, versions « complètes » seulement ; ni App Store, ni Android
F	Réseau HTTPS général	Catalogue AniList, extensions, images, flux	Système : URLSession / App Transport Security (iOS) ; pile TLS d'Android	Toutes les versions
G	Mises à jour à la volée (expo-updates)	Vérification de signature des mises à jour	expo-updates (RSA PKCS#1 v1.5 / SHA-256, vérification seulement)	Désactivé dans la version 1.0.0 (aucun serveur ni certificat configuré)

3. Description détaillée par fonction

A. Couche sociale pair à pair (libsodium)

Le code s'exécute dans un environnement JavaScript séparé (« worklet » Bare, react-native-bare-kit 0.15.6) : src/p2p/worklet/. Toutes les primitives proviennent de **libsodium 1.0.21**, compilée dans le module natif sodium-native 5.1.0.

Bibliothèques HoleyPunch utilisées (versions résolues) : hyperswarm 4.17.2, hyperdht 6.34.0, @hyperswarm/secret-stream 6.9.2, noise-handshake 4.2.0, noise-curve-ed 2.1.0, sodium-secretstream 1.2.0, hypercore 11.37.1, hypercore-crypto 3.7.0, corestore 7.13.0, autobase 7.28.2, hyperbee 2.27.3, protomux 3.12.1, blind-peering 2.10.0, blind-pairing 2.3.1 / blind-pairing-core 2.10.1, keet-identity-key 3.2.0, sodium-hmac 2.1.0, bip39-mnemonic 2.5.0.

A.1 Chiffrement du transport entre pairs (confidentialité)

Élément	Détail
Établissement de session	Protocole Noise (noise-handshake). Connexions via HyperDHT : motif IK (hyperdht/lib/noise-wrap.js). Motif par défaut de secret-stream : XX . Nom complet du protocole : Noise_IK_Ed25519_ChaChaPoly_BLAKE2b / Noise_XX_Ed25519_ChaChaPoly_BLAKE2b
Échange de clés	Diffie-Hellman sur Edwards25519 (crypto_scalarmult_ed25519_noclamp, noise-curve-ed), clés de 256 bits, niveau de sécurité ≈ 128 bits
Chiffrement de la poignée de main	ChaCha20-Poly1305 IETF (crypto_aead_chacha20poly1305_ietf), clé de 256 bits, tag de 128 bits
Hachage / dérivation	BLAKE2b-512, HMAC-BLAKE2b, HKDF (selon la spécification Noise)
Chiffrement du flux	XChaCha20-Poly1305 « secretstream » de libsodium (crypto_secretstream_xchacha20poly1305), clé de session de 256 bits, une par sens

Élément	Détail
Authentification des pairs	Clé publique statique Ed25519 de chaque pair, authentifiée par la poignée de main Noise

Tout le trafic entre deux appareils Huwa, ou entre un appareil et un relais, est chiffré ainsi. Le chiffrement est toujours actif et ne peut pas être désactivé.

A.2 Signatures et intégrité des journaux (authentification / intégrité)

Élément	Détail
Journaux (Hypercore / Autobase)	Chaque bloc est rattaché à un arbre de Merkle BLAKE2b-256 dont la racine est signée en Ed25519 (crypto_sign_detached) par la clé de l'écrivain
Identité	Chaîne d'attestation keet-identity-key : la clé d'identité Ed25519 signe la clé de chaque appareil ; la clé d'appareil signe l'association « écrivain ↔ base personnelle » (src/p2p/worklet/auth.js)
Clés de discussion	Les identifiants des salons publics et des conversations sont des clés publiques Ed25519 dérivées de façon déterministe d'un condensat BLAKE2b-256 (src/p2p/worklet/node.js)
Anti-spam	Preuve de travail : BLAKE2b-256 sur la charge utile et un nonce de 32 bits (src/p2p/worklet/pow.js)
Empreintes	BLAKE2b (crypto_generichash), 128 ou 256 bits

Ces fonctions ne protègent pas la confidentialité : les commentaires publics, le profil et la progression sont des données **signées, en clair**.

A.3 Messages privés (confidentialité de bout en bout)

Élément	Détail
Primitive	crypto_box_seal de libsodium (« sealed box ») : clé éphémère X25519 propre à chaque message, puis XSalsa20-Poly1305 ; clé de 256 bits, tag de 128 bits (src/p2p/worklet/seal.js)
Clés de réception	Paire X25519 « boîte » de l'utilisateur (crypto_box_seed_keypair), dérivée d'une graine de 256 bits
Envoi	Chaque message est scellé deux fois : pour la boîte du destinataire et pour celle de l'expéditeur (node.js, envoi de message privé)
Rotation	Si un appareil est révoqué, une nouvelle graine de boîte aléatoire (256 bits) est créée puis scellée (crypto_box_seal) pour chacun des appareils restants, dont les clés Ed25519 sont converties en X25519 (crypto_sign_ed25519_pk_to_curve25519)

Seuls les deux interlocuteurs peuvent déchiffrer. Un relais qui réplique la conversation ne voit que du chiffré.

A.4 Appairage d'un nouvel appareil (confidentialité)

blind-pairing-core 2.10.1 : l'invitation (QR code) contient une graine ; la requête et la réponse sont chiffrées en **XChaCha20-Poly1305 IETF** (crypto_aead_xchacha20poly1305_ietf, clé de 256 bits) et signées en **Ed25519**. La réponse transmet au nouvel appareil la graine de boîte des messages privés.

A.5 Dérivation des clés à partir de la phrase de récupération

Étape	Algorithme
Génération	256 bits d'aléa (randombytes_buf) → phrase BIP-39 de 24 mots (bip39-mnemonic)
Graine	PBKDF2-HMAC-SHA-512 , 2 048 itérations, sel "mnemonic" (norme BIP-39) → 512 bits

Étape	Algorithme
Clé d'identité	Dérivation hiérarchique HMAC-SHA-512 (indices renforcés) → graine Ed25519 de 256 bits (keet-identity-key)
Clés symétriques	HMAC-SHA-512 avec des étiquettes (huwa/box/v1, huwa/box-rotate/v1) → graines de 256 bits des boîtes X25519
Clés d'appareil	Paires Ed25519 aléatoires (crypto_sign_keypair)

A.6 Relais (« blind peers »)

Les relais sont des nœuds Hypercore toujours en ligne (logiciel libre `blind-peer`), auxquels l'application demande de conserver ses journaux (`src/p2p/worklet/relay.js`). Leurs clés publiques sont fixées dans la configuration ou ajoutées par l'utilisateur. Ce qu'ils voient :

- **messages privés** : du chiffré uniquement (§ A.3) ;
- **autres journaux** (profil, appareils, listes, progression, commentaires publics) : stockés **signés mais non chiffrés**. Le chiffrement au repos d'Hypercore / Autobase existe dans les bibliothèques mais **n'est pas activé** dans la version 1.0.0 ;
- la liaison appareil ↔ relais est chiffrée comme tout transport (§ A.1).

Les relais sont exploités par l'éditeur ou par des tiers. Ils ne font pas partie du produit déclaré. L'application ne contient pas de code de relais.

B. Sauvegarde de la phrase de récupération dans une clé d'accès (passkey)

Code : `src/p2p/passkey-core.ts`, `src/p2p/passkey.ts`, `modules/huwa-passkey/ios/HuwaPasskeyModule.swift`.
Disponible sur iOS 17 et ultérieur ; non disponible sur Android.

Étape	Détail	Fournisseur
Clé d'accès	WebAuthn (AuthenticationServices, ASAAuthorizationPlatformPublicKeyCredentialProvider), domaine <code>huwa.mciut.fr</code> . Paire de clés générée et conservée par le gestionnaire de mots de passe (ECDSA P-256 en général)	Système / gestionnaire de mots de passe
Extension PRF (iOS 18+)	Sortie pseudo-aléatoire de 256 bits liée à la clé d'accès, avec un sel fixe SHA-256 ("huwa-passkey-v1")	Système / gestionnaire
Dérivation	HKDF-SHA-256 (sel <code>huwa-passkey-v1</code> , info <code>huwa/passkey/phrase-key/v1</code>) → clé de 256 bits	@noble/hashes 2.4.0, embarqué
Chiffrement	AES-256-GCM , nonce aléatoire de 96 bits, tag de 128 bits, données associées <code>huwa-passkey-v1</code>	CryptoKit (AES.GCM) via <code>expo-crypto</code> 57.0.3 : système
Stockage	Extension WebAuthn largeBlob de la clé d'accès (≤ 1 024 octets)	Système / gestionnaire
Sans PRF (iOS 17, ou gestionnaire sans PRF)	La phrase est déposée dans le <code>largeBlob</code> sans chiffrement applicatif . Elle n'est alors protégée que par le gestionnaire de mots de passe (chiffrement de bout en bout du fournisseur, déblocage par Face ID / code)	Système / gestionnaire
Aléa	<code>SecRandomCopyBytes</code> (iOS), <code>SecureRandom</code> (Android), via <code>expo-crypto</code>	Système

C. Stockage des secrets sur l'appareil et dans le Trousseau iCloud

Donnée	Stockage	Protection
Phrase de récupération (locale)	expo-secure-store : Trousseau iOS (WHEN_UNLOCKED_THIS_DEVICE_ONLY) ; Android Keystore	Chiffrement du système
Copie synchronisée de la phrase (iOS, option activée par défaut, désactivable)	Élément de Trousseau kSecAttrSynchronizable, service app.huwa.cloud-backup (modules/huwa-keychain)	Trousseau iCloud , chiffré de bout en bout par Apple
Clés de l'appareil, graines de boîte	Base locale Hyperbee dans le conteneur de l'application, exclue des sauvegardes iCloud / Finder	Protection des données du système (chiffrement du stockage) ; pas de chiffrement applicatif supplémentaire

L'application n'implémente pas elle-même ces chiffrements : elle appelle les interfaces du système.

D. Moteur BitTorrent natif (Rust), versions « complètes » seulement

Code : native/huwa-torrent-core (bibliothèque Rust liée à l'application par modules/huwa-torrent). Il n'est compilé que si HUWA_TORRENT=1. La version App Store est compilée avec HUWA_TORRENT=0 (voir eas.json, profil store) : elle ne contient pas le moteur. Dans les autres versions, le moteur est désactivé par défaut et l'utilisateur doit l'activer.

Élément	Détail
Bibliothèque	librqbit 9.0.1 (copie locale avec correctifs non cryptographiques, native/vendor/librqbit/HUWA_PATCHES.md), options rust-tls et disable-upload ; default-features = false (pas d'OpenSSL)
TLS	rustls 0.23.45 , fournisseur cryptographique aws-lc-rs 1.18.1 (aws-lc-sys 0.45.0). Vérification des certificats : rustls-platform-verifier 0.7.1 (magasin de certificats du système), rustls-native-certs 0.8.4
Usages	Client HTTPS : trackers BitTorrent en https:// ; proxy de lecture anticipée vers des liens HTTPS distants (src/http_proxy.rs, reqwest 0.13.5, HTTP/1.1). Sur Android, les flux HTTPS ne passent pas par ce proxy
Versions et suites TLS	TLS 1.2 et 1.3 (configuration par défaut de rustls) : AES-128-GCM, AES-256-GCM, ChaCha20-Poly1305 ; échange ECDHE (X25519, P-256, P-384) ; signatures serveur ECDSA, RSA, Ed25519 (vérification seulement)
Intégrité BitTorrent	SHA-1 des morceaux (BitTorrent v1), via aws-lc-rs (librqbit-sha1-wrapper). Usage d'intégrité, pas de sécurité
Chiffrement de protocole BitTorrent	Absent. librqbit 9.0.1 n'implémente pas le chiffrement de flux MSE/PE (RC4) : aucune occurrence dans le code de librqbit, librqbit-peer-protocol et librqbit-core, et aucune bibliothèque RC4 dans Cargo.lock. Les échanges avec les pairs BitTorrent sont en clair
Autres	chacha20 0.10.2 n'est présent que comme générateur pseudo-aléatoire de la bibliothèque rand (pas de chiffrement)

E. Lecteur de secours libmpv (iOS, versions « complètes » seulement)

modules/huwa-mpv/ios/Libmpv.xcframework (MPVKit 1.0.0, variante LGPL : mpv 0.41.0, FFmpeg n8.1.2). Il n'est pas lié dans la version App Store (HUWA_LITE=1 → huwa.mpv = 0) et n'existe pas sur Android.

Élément	Détail
TLS	GnuTLS 3.8.11 (lié statiquement ; cryptographie GnuTLS / Nettle), utilisé par FFmpeg pour les flux https:// ; TLS 1.2 / 1.3, suites TLS_AES_128_GCM_SHA256, TLS_AES_256_GCM_SHA384, TLS_CHACHA20_POLY1305_SHA256 et suites TLS 1.2 correspondantes

Élément	Détail
HLS chiffré	Protocole crypto de FFmpeg : déchiffrement AES-128-CBC des segments HLS (méthode standard EXT-X-KEY METHOD=AES-128) quand la source le prévoit. Déchiffrement seul, clé fournie par la source

F. HTTPS via le système

Toutes les requêtes réseau ordinaires (catalogue AniList, extensions, images, lecture via AVPlayer / ExoPlayer) passent par les piles réseau du système : URLSession avec App Transport Security sur iOS, pile TLS du système sur Android (OkHttp sur la plateforme). L'application ne fournit aucune implémentation TLS pour ces échanges.

G. Mises à jour à la volée (expo-updates)

Le mécanisme prévoit la **vérification** d'une signature RSA PKCS#1 v1.5 / SHA-256 (app.json, codeSigningMetadata). Il est **désactivé** dans la version 1.0.0 (app.config.js). Il s'agit d'authentification uniquement.

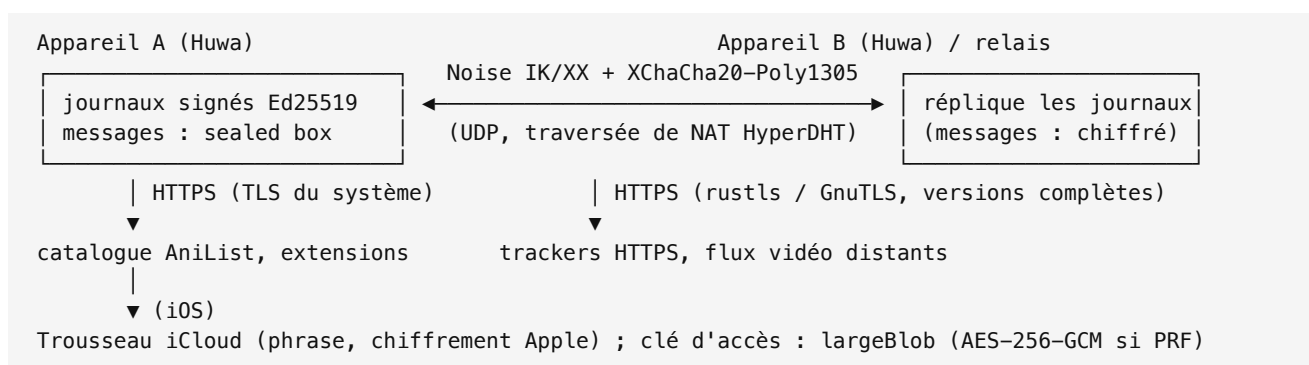
4. Tableau récapitulatif des algorithmes

Algorithme	Fonction	Taille de clé / paramètre	Bibliothèque	Embarqué ou système
XChaCha20-Poly1305 (secretstream)	Chiffrement du transport P2P	256 bits	libsodium 1.0.21	Embarqué
ChaCha20-Poly1305 IETF	Poignée de main Noise	256 bits	libsodium 1.0.21	Embarqué
XChaCha20-Poly1305 IETF	Appairage d'appareils	256 bits	libsodium 1.0.21	Embarqué
X25519 + XSalsa20-Poly1305 (crypto_box_seal)	Messages privés, partage de graine	256 bits	libsodium 1.0.21	Embarqué
DH Edwards25519 (Noise)	Accord de clé de transport	256 bits	libsodium 1.0.21	Embarqué
Ed25519	Signatures (journaux, identité, appairage)	256 bits	libsodium 1.0.21 ; @noble/curves 2.4.0 (mode local, hors worklet)	Embarqué
BLAKE2b (256 / 512 bits), HMAC-BLAKE2b	Hachage, Merkle, dérivation Noise, preuve de travail	—	libsodium 1.0.21 ; @noble/hashes 2.4.0	Embarqué
PBKDF2-HMAC-SHA-512 (2 048 it.)	Phrase → graine (BIP-39)	512 bits	libsodium 1.0.21 (extension sodium-native)	Embarqué
HMAC-SHA-512	Dérivation hiérarchique de clés	256 bits produits	sodium-hmac 2.1.0 sur libsodium 1.0.21	Embarqué
HKDF-SHA-256	Clé de sauvegarde passkey	256 bits	@noble/hashes 2.4.0	Embarqué
AES-256-GCM	Chiffrement de la sauvegarde passkey	256 bits, nonce 96 bits	CryptoKit via expo-crypto	Système
WebAuthn (PRF, largeBlob)	Clé d'accès	Selon le gestionnaire (ECDSA P-256)	AuthenticationServices	Système

Algorithme	Fonction	Taille de clé / paramètre	Bibliothèque	Embarqué ou système
Trousseau iOS / iCloud, Android Keystore	Stockage de secrets	—	Security.framework, Android Keystore	Système
TLS 1.2 / 1.3 (AES-GCM, ChaCha20-Poly1305, ECDHE)	HTTPS général	128 / 256 bits	URLSession, pile Android	Système
TLS 1.2 / 1.3 (AES-GCM, ChaCha20-Poly1305, ECDHE)	HTTPS du moteur torrent	128 / 256 bits	rustls 0.23.45 + aws-lc-rs 1.18.1	Embarqué (versions complètes)
TLS 1.2 / 1.3	HTTPS du lecteur libmpv	128 / 256 bits	GnuTLS 3.8.11	Embarqué (iOS, versions complètes)
AES-128-CBC (déchiffrement)	Segments HLS chiffrés	128 bits	FFmpeg n8.1.2	Embarqué (iOS, versions complètes)
SHA-1	Intégrité des morceaux BitTorrent	—	aws-lc-rs	Embarqué (versions complètes)
SHA-256	Sel PRF, identifiants	—	@noble/hashes	Embarqué
RSA PKCS#1 v1.5 / SHA-256	Vérification des mises à jour (désactivée)	Selon le certificat	expo-updates	Embarqué, inactif

Aléa : randombytes_buf de libsodium (appuyé sur le générateur du système), SecRandomCopyBytes (iOS), SecureRandom (Android).

5. Flux de données protégés



- Appareil ↔ appareil / relais** : transport toujours chiffré (§ A.1). Contenu des messages privés chiffré de bout en bout (§ A.3). Le reste des journaux est signé.
- Appareil ↔ services web** : HTTPS via le système, ou via rustls / GnuTLS dans les versions complètes.
- Appareil ↔ Apple / gestionnaire de mots de passe** : sauvegarde de la phrase (Trousseau iCloud ; clé d'accès, chiffrée en AES-256-GCM quand PRF est disponible).

6. Gestion des clés

- Génération** : toutes les clés sont générées sur l'appareil de l'utilisateur. L'éditeur ne génère, ne détient, ne séquestre ni ne peut recouvrer aucune clé.

- **Clés racine** : dérivées de la phrase de récupération (§ A.5). La phrase est stockée dans le Trousseau ou le Keystore du système.
 - **Clés de session** : éphémères, issues de la poignée de main Noise, jamais stockées.
 - **Révocation** : un appareil peut être révoqué ; la graine de boîte des messages privés est alors renouvelée (§ A.3).
 - **Pas de serveur de clés**, pas d'autorité de certification propre à Huwa.
-

7. Possibilités de modification

- Les algorithmes, tailles de clés et protocoles sont figés dans le code. Aucun réglage ne permet de les changer.
 - L'utilisateur peut seulement activer ou désactiver certaines fonctions (sauvegarde iCloud, clé d'accès, relais, moteur torrent dans les versions complètes). Il ne peut pas changer les mécanismes cryptographiques.
 - Les extensions ajoutées par l'utilisateur sont des sources de contenu (catalogues, liens). Elles n'ont accès à aucune fonction cryptographique de l'application.
 - Le code source des bibliothèques cryptographiques utilisées est public : libsodium, rustls, aws-lc, GnuTLS, @noble/hashes, bibliothèques Holepunch.
-

8. Points d'attention signalés par le fournisseur

- Le chiffrement au repos des journaux sur les relais n'est **pas** activé, sauf pour le contenu des messages privés (§ A.6).
- Sans PRF, la sauvegarde par clé d'accès n'a pas de chiffrement applicatif (§ B).
- Les composants D et E (rustls / aws-lc-rs, GnuTLS, FFmpeg) ne sont présents que dans les versions distribuées hors App Store.